EXPECTED ANSWERS FOR PYTHON PRACTICAL VIVA

Study Notes for Students | 110 Oral Examination Questions

Instructions: Read each answer and understand the concept. For questions about your mini project, prepare an answer based on the code you actually developed. Examples are illustrative.

Part 1: Python Fundamentals and Operators

1. What is Python?

Answer: Python is a high-level, interpreted, general-purpose programming language known for its readable and simple syntax.

2. What are the main features of Python?

Answer: Python is easy to learn, interpreted, high-level, portable, open-source, object-oriented, and has a large standard library.

3. What is the purpose of the print() function?

Answer: print() displays text, values, or the result of an expression on the screen. Example: print('Hello').

4. What is the purpose of the input() function?

Answer: input() accepts data from the user through the keyboard and returns it as a string. Example: name = input('Enter name: ').

5. What is a variable?

Answer: A variable is a name used to refer to a value in a program. Example: age = 18.

6. What are the basic data types in Python?

Answer: Common basic types are int (integer), float (decimal number), str (text), bool (True or False), and NoneType (no value).

7. Why is indentation important in Python?

Answer: Indentation defines blocks of code, such as the body of an if statement, loop, or function. Incorrect indentation can cause an error or change program structure.

8. What is Python syntax?

Answer: Syntax is the set of rules for writing valid Python statements and expressions.

9. What are arithmetic operators?

Answer: Arithmetic operators perform mathematical calculations: +, -, *, /, //, %, and **.

10. What is the difference between / and // operators?

Answer: / performs division and normally returns a floating-point result; // performs floor division, rounding the result down to the nearest whole value. Example: 7 / 2 = 3.5; 7 // 2 = 3.

11. What is the % operator used for?

Answer: The modulo operator % returns the remainder after division. Example: 7 % 2 gives 1.

12. What are relational operators?

Answer: They compare values and return True or False: ==, !=, >, <, >=, and <=.

13. What are logical operators?

Answer: and, or, and not combine or reverse Boolean conditions.

14. What is the difference between = and ==?

Answer: = assigns a value to a variable, while == compares two values for equality.

Part 2: Conditional Statements and Loops

15. What is a conditional statement?

Answer: A conditional statement chooses which code to execute depending on whether a condition is True or False.

16. Explain the if statement.

Answer: An if statement executes its indented block only when its condition is True. Example: if age >= 18: print('Eligible').

17. Explain the if-else statement.

Answer: if-else selects between two blocks: the if block runs when the condition is True; otherwise, the else block runs.

18. What is a nested if statement?

Answer: A nested if is an if statement placed inside another if or else block.

19. What is the difference between nested if and if-elif-else?

Answer: Nested if checks a condition inside another condition. if-elif-else checks multiple alternative conditions in sequence and is often easier to read for a list of choices.

20. What is a loop?

Answer: A loop repeats a block of code while iterating over items or while a condition remains True.

21. What is the difference between for and while loops?

Answer: A for loop commonly iterates over a sequence or range. A while loop repeats as long as its condition is True.

22. What is an accumulator variable?

Answer: An accumulator stores a running total or combined result. Example: total = total + mark.

23. What can cause an infinite loop?

Answer: A loop may run forever if its stopping condition never becomes False, often because the variable controlling it is not updated correctly.

24. What is the use of the break statement?

Answer: break immediately exits the nearest enclosing loop.

25. What is a nested loop?

Answer: A nested loop is a loop placed inside another loop.

26. Why are nested loops used in pattern generation?

Answer: The outer loop usually controls rows, while the inner loop controls the characters or symbols printed in each row.

Part 3: Strings

27. What is a string?

Answer: A string is a sequence of characters enclosed in quotes, such as 'Python' or "Hello".

28. What is string indexing?

Answer: Indexing accesses one character by its position. Python uses zero-based indexing, so text[0] is the first character; negative indexes count from the end.

29. What is string slicing?

Answer: Slicing extracts part of a string using text[start:stop:step]. The stop position is excluded.

30. What is string traversal?

Answer: String traversal means visiting or processing each character, commonly using a for loop.

31. How can the number of characters in a string be counted?

Answer: Use len(text), which returns the number of characters, including spaces and punctuation.

32. How can words in a string be counted?

Answer: For ordinary whitespace-separated words, use len(text.split()). This treats consecutive spaces as one separator.

33. How can vowels be counted in a string?

Answer: Loop through the characters and count those found in 'aeiouAEIOU'.

34. What is string formatting?

Answer: String formatting inserts values into a string in a readable form. Example: f'Name: {name}, Age: {age}'.

35. What are built-in string methods?

Answer: They are methods available on strings for tasks such as changing case, removing surrounding spaces, replacing text, and splitting text.

36. What is the use of upper()?

Answer: upper() returns a new string with alphabetic characters converted to uppercase. Example: 'hello'.upper() gives 'HELLO'.

37. What is the use of lower()?

Answer: lower() returns a new string with alphabetic characters converted to lowercase.

38. What is the use of strip()?

Answer: strip() removes whitespace from the beginning and end of a string by default.

39. What is the use of replace()?

Answer: replace(old, new) returns a string in which occurrences of old text are replaced by new text.

40. What is the use of split()?

Answer: split() divides a string into a list of pieces, using whitespace by default or a supplied separator.

Part 4: Lists

41. What is a list?

Answer: A list is an ordered, mutable collection written with square brackets, for example marks = [70, 80, 90].

42. What is list indexing?

Answer: List indexing accesses an element by position, starting at index 0. Example: marks[0] returns the first item.

43. How can an element be added to a list?

Answer: Use append(value) to add at the end, insert(index, value) to add at a position, or extend(iterable) to add multiple items.

44. What is the difference between append() and insert()?

Answer: append(value) adds an item at the end; insert(index, value) adds it at a specified position.

45. How can an element be removed from a list?

Answer: Use remove(value) to remove the first matching value, pop(index) to remove and return an item by index, or del to delete by index or slice.

46. What is list traversal?

Answer: List traversal means visiting each list element, often with a for loop.

47. Are lists mutable?

Answer: Yes. Lists are mutable, so their elements can be changed after the list is created.

48. What is sorting?

Answer: Sorting arranges elements in an order, such as ascending or descending.

49. What is searching?

Answer: Searching is the process of finding whether a value exists in a collection and, if needed, locating its position.

50. What is the difference between sort() and sorted()?

Answer: list.sort() changes the original list and returns None. sorted(iterable) returns a new sorted list and leaves the original iterable unchanged.

51. How can a list be sorted in descending order?

Answer: Use my_list.sort(reverse=True) to change the list, or sorted(my_list, reverse=True) to create a new list.

52. What is a nested list?

Answer: A nested list is a list containing one or more lists as elements, such as [[1, 2], [3, 4]].

53. How can a specific element be accessed from a nested list?

Answer: Use two indexes. For example, data[1][0] accesses the first element of the second inner list.

54. What is two-dimensional indexing?

Answer: It uses a row index and a column index to access an element in a two-dimensional structure, for example matrix[row][column].

55. What is a tuple?

Answer: A tuple is an ordered collection, usually written with parentheses, such as (18.5, 73.8).

Part 5: Tuples and Dictionaries

56. What is the difference between a list and a tuple?

Answer: Lists are mutable and use square brackets; tuples are immutable and commonly use parentheses.

57. Are tuples mutable?

Answer: No. Tuples are immutable: their elements cannot be reassigned after creation, although a tuple can contain a mutable object.

58. Why are tuples useful for storing fixed coordinates?

Answer: A tuple is suitable for coordinates that should remain grouped and unchanged, such as (latitude, longitude).

59. What is a dictionary?

Answer: A dictionary stores key-value pairs, for example student = {'name': 'Asha', 'age': 18}.

60. What are keys and values in a dictionary?

Answer: A key identifies an entry, and its value is the data associated with that key.

61. How can a dictionary value be accessed?

Answer: Use dictionary[key], such as student['name'], or dictionary.get(key).

62. How can a new key-value pair be added?

Answer: Assign a value to a new key, for example student['city'] = 'Pune'.

63. How can a dictionary value be updated?

Answer: Assign a new value to an existing key, for example student['age'] = 19.

64. Can dictionary keys be duplicated?

Answer: No. Keys are unique. Assigning a value to an existing key updates that key's value.

65. What is the use of the get() method?

Answer: get(key, default) returns the value for a key, or the default if the key is absent, without raising a KeyError.

66. What is word frequency?

Answer: Word frequency is the number of times each word appears in a text.

67. How can a dictionary be used to count word frequency?

Answer: Split the text into words and increase a dictionary count for each word. Example: freq[word] = freq.get(word, 0) + 1.

68. What is a user-defined function?

Answer: A function written by the programmer to perform a particular task and reuse that code.

Part 6: Functions, Modules and Random/Math

69. How is a function defined in Python?

Answer: Use def, followed by the function name, parentheses, and a colon. Example: def greet(): followed by an indented body.

70. What is the difference between parameters and arguments?

Answer: Parameters are names in a function definition; arguments are actual values supplied when calling the function.

71. What is a return value?

Answer: A return value is the result a function sends back to the code that called it using return.

72. What is the difference between print() and return?

Answer: print() displays information on the screen. return sends a value back to the caller so it can be stored or used in further calculations.

73. What is modular programming?

Answer: Modular programming divides a program into smaller, manageable parts such as functions and modules.

74. What is a menu-driven program?

Answer: It displays a set of options and performs an action based on the user's selection, often repeating until the user chooses to exit.

75. What is a module in Python?

Answer: A module is a Python file containing reusable code, such as functions, variables, or classes.

76. How do you import a module?

Answer: Use import module_name, for example import math, or import a specific item with from math import sqrt.

77. What is the random module?

Answer: The random module provides functions for generating pseudo-random values and making random selections.

78. What is the use of random.randint()?

Answer: random.randint(a, b) returns a pseudo-random integer from a through b, including both endpoints.

79. What is the math module?

Answer: The math module provides mathematical functions and constants, such as sqrt(), ceil(), floor(), and pi.

Part 7: File Handling

80. What is file handling?

Answer: File handling means creating, opening, reading, writing, appending to, and closing files to store or retrieve data.

81. What is the use of the open() function?

Answer: open() opens a file and returns a file object for reading or writing. Example: open('data.txt', 'r').

82. What are file modes?

Answer: File modes specify how a file is opened, such as for reading, writing, appending, or binary access.

83. What is the difference between r, w, and a modes?

Answer: r reads an existing file; w writes and truncates an existing file or creates a new one; a appends to the end or creates a new file.

84. What is the use of read()?

Answer: read() reads file content and returns it as a string in text mode, unless a size is specified.

85. What is the use of write()?

Answer: write() writes a string to a file and returns the number of characters written.

86. What is the advantage of using with open()?

Answer: It automatically closes the file when the with block ends, including when an exception occurs.

Part 8: Exception Handling and Input Validation

87. What is exception handling?

Answer: Exception handling lets a program respond to runtime problems without ending unexpectedly, using constructs such as try and except.

88. What is an exception?

Answer: An exception is an error or unusual event raised while a program is running, such as dividing by zero or converting invalid text to an integer.

89. What is the purpose of try-except?

Answer: The try block contains code that may raise an exception; the except block handles a matching exception.

90. What is input validation?

Answer: Input validation checks that user-provided data is acceptable, such as ensuring a mark is within a valid range or a value is an integer.

91. What is the difference between a syntax error and a runtime error?

Answer: A syntax error means code breaks Python's grammar and cannot be parsed. A runtime error occurs while validly parsed code is executing.

92. What is the purpose of the finally block?

Answer: finally contains cleanup code that runs whether an exception occurs or not, such as releasing a resource.

Part 9: Mini Projects and General Python Concepts

93. How are questions stored in an online quiz system?

Answer: Questions can be stored in a list of dictionaries or similar structures, with fields for the question, options, and correct answer.

94. How are answers checked in a quiz program?

Answer: The program compares the user's selected answer with the stored correct answer, using an if statement or equivalent logic.

95. How is the final score calculated?

Answer: The program starts a score counter at zero and increases it for each correct answer, then displays the total.

96. How can lists and dictionaries be used in an attendance system?

Answer: A list can store student records or student names, while dictionaries can map each student's name or ID to an attendance status.

97. How can data be stored permanently using files?

Answer: Write the data to a file and read it again when needed. Data in variables alone is normally lost when the program ends.

98. Which data structures are used in your mini project?

Answer: Answer according to your own code. For example, a quiz may use lists and dictionaries; a phonebook may use a dictionary; an attendance system may use lists and dictionaries.

99. Which functions are used in your mini project?

Answer: Name the functions actually present in your code and briefly explain each function's purpose. Do not list functions that your project does not use.

100. How is exception handling implemented in your mini project?

Answer: Explain the actual try-except blocks in your code, the errors they handle, and the message or recovery action used.

101. What modules are used in your mini project?

Answer: Name only the modules imported in your code, such as random, math, or a module you created yourself. If none are imported, say so.

102. What real-world problem does your mini project solve?

Answer: Describe the practical task your program helps with, such as recording attendance, storing student details, managing contacts, or conducting a quiz.

103. What challenges did you face while developing the project?

Answer: Give an honest example from your work, such as handling invalid input, fixing a logic error, or saving and retrieving file data, and explain how you solved it.

104. How can your project be improved in the future?

Answer: Possible improvements include a graphical interface, stronger input validation, search and update options, database storage, user authentication, and better reporting—choose improvements relevant to your project.

105. What is the difference between a syntax error, runtime error, and logical error?

Answer: A syntax error breaks Python grammar; a runtime error occurs during execution; a logical error allows the program to run but produces an incorrect result.

106. What is type casting?

Answer: Type casting converts a value from one data type to another, such as int('12'), float('3.5'), or str(25).

107. What are mutable and immutable data types?

Answer: Mutable objects can be changed after creation, such as lists and dictionaries. Immutable objects cannot have their value changed in place, such as integers, strings, and tuples.

108. What is the difference between int, float, and str?

Answer: int represents whole numbers, float represents numbers with a fractional part, and str represents text.

109. What is the difference between break, continue, and pass?

Answer: break exits the loop; continue skips the rest of the current iteration and moves to the next; pass does nothing and acts as a placeholder.

110. What is the purpose of comments in Python?

Answer: Comments explain code for readers and are ignored by Python during execution. A single-line comment begins with #.